WSD Lab-7

Aim : Study of Message Contract in WCF

1. Implement WCF service with a message contract for a Book Shop discussed in class.

=>

IServiceClass.cs :

```
using System.ServiceModel;

namespace WcfServiceLibraryBookShop
{
   [ServiceContract]
   public interface IServiceClass
   {
      [OperationContract]
      string InitiateOrder();
      [OperationContract]
      BookOrder PlaceOrder(BookOrder request);
      [OperationContract]
      string FinalizeOrder();
   }
   [MessageContract]
   public class BookOrder
   {
      private string isbn, firstname, lastname, address, ordernumber;
      private int quantity;
      public BookOrder() { }
      public BookOrder(BookOrder message)
      {
         this.isbn = message.isbn;
         this.firstname = message.firstname;
         this.lastname = message.lastname;
         this.address = message.address;
         this.quantity = message.quantity;
      }
      [MessageHeader]
      public string ISBN { get { return isbn; } set { isbn = value; } }
```

```
        [MessageBodyMember]
        public string Firstname { get { return firstname; } set { firstname = value; } }
        [MessageBodyMember]
        public string Lastname { get {return lastname; } set { lastname = value; } }
        [MessageBodyMember]
        public string Address { get { return address; } set { address = value; } }
        [MessageBodyMember]
        public string OrderNumber { get { return ordernumber; } set { ordernumber = value; } }
        [MessageBodyMember]
        public int Quantity { get { return quantity; } set { quantity = value; } }
    }
}
```

ServiceClass.cs :

```
namespace WcfServiceLibraryBookShop
{
    public class ServiceClass : IServiceClass
    {
        public string FinalizeOrder()
        {
            return "Order placed successfully";
        }
        public string InitiateOrder()
        {
            return "Initiating Order...";
        }
        public BookOrder PlaceOrder(BookOrder request)
        {
            BookOrder order = new BookOrder(request);
            order.OrderNumber = "12345678";
            return order;
        }
    }
}
```

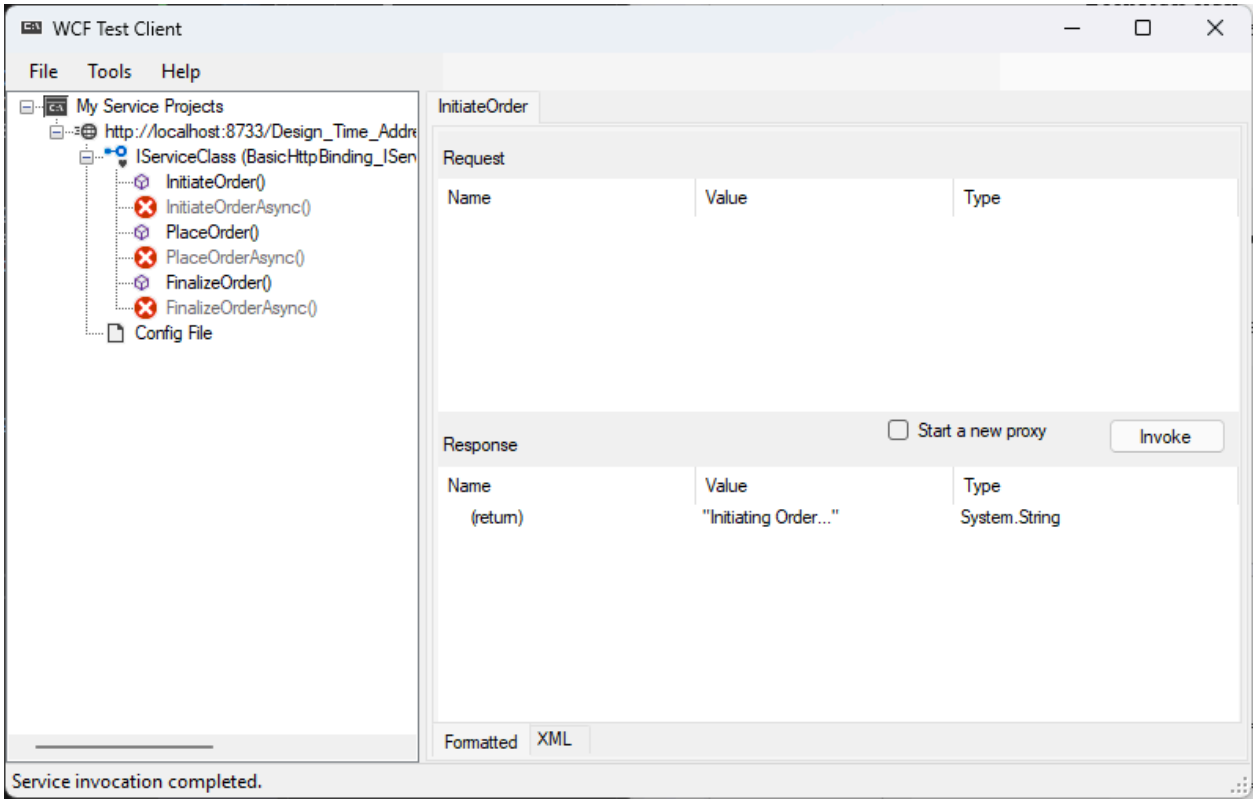
App.config :

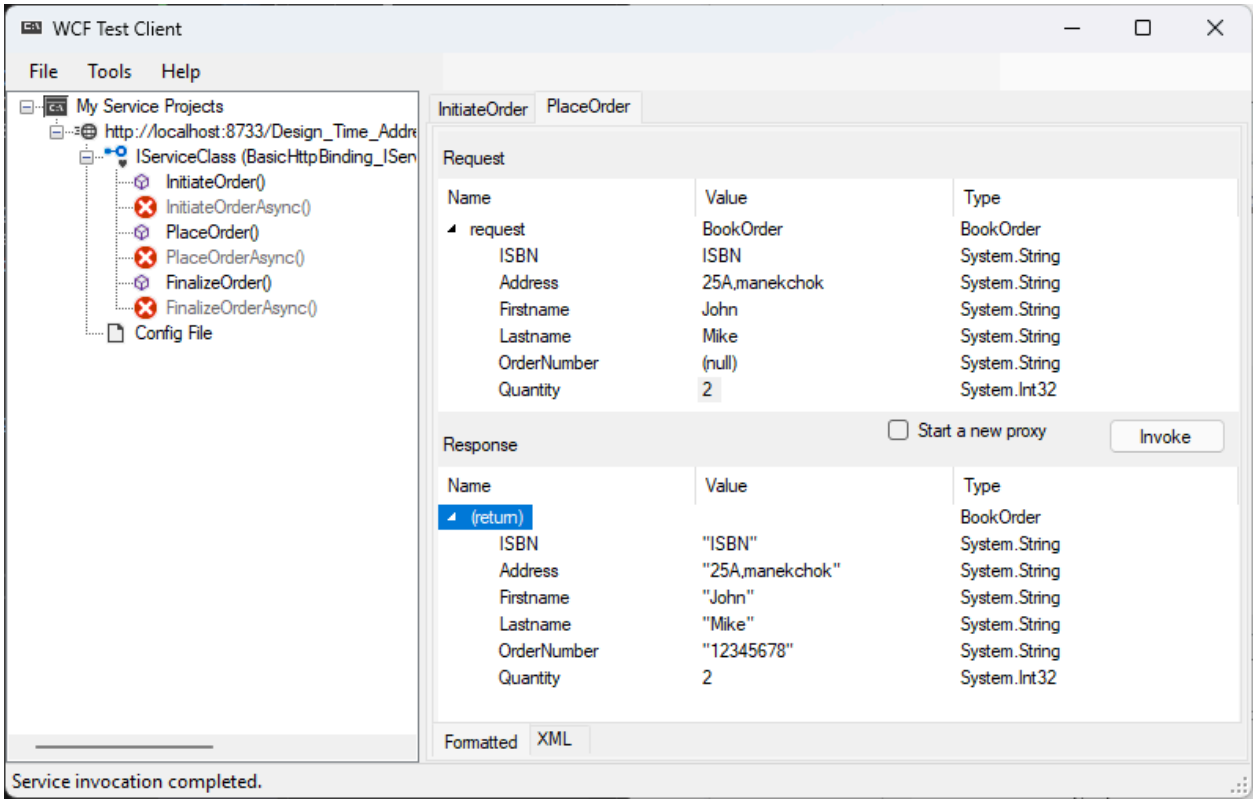
```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
    <appSettings>
        <add key="aspnet:UseTaskFriendlySynchronizationContext" value="true" />
```

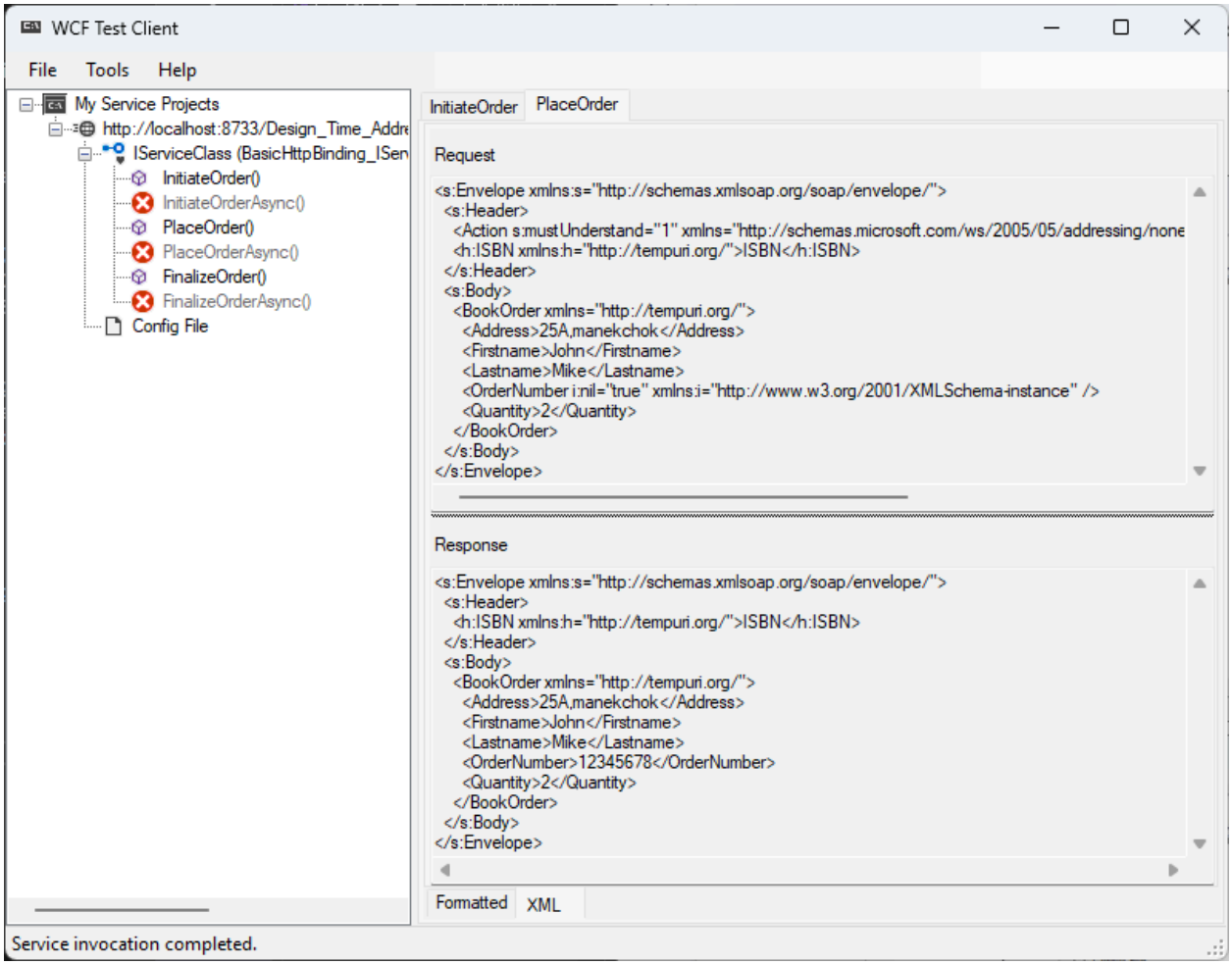
```
  </appSettings>
  <system.web>
    <compilation debug="true" />
  </system.web>
  <system.serviceModel>
    <services>
      <service name="WcfServiceLibraryBookShop.ServiceClass">
        <host>
          <baseAddresses>
            <add baseAddress =
"http://localhost:8733/Design_Time_Addresses/WcfServiceLibraryBookShop/ServiceClass/" />
          </baseAddresses>
        </host>
        <endpoint address="" binding="basicHttpBinding"
        contract="WcfServiceLibraryBookShop.IServiceClass">
          <identity>
            <dns value="localhost"/>
          </identity>

        </endpoint>
        <endpoint address="mex" binding="mexHttpBinding"
        contract="IMetadataExchange"/>
      </service>
    </services>
    <behaviors>
      <serviceBehaviors>
        <behavior>
          <serviceMetadata httpGetEnabled="True" httpsGetEnabled="True"/>
          <serviceDebug includeExceptionDetailInFaults="False" />
        </behavior>
      </serviceBehaviors>
    </behaviors>
  </system.serviceModel>
</configuration>
```

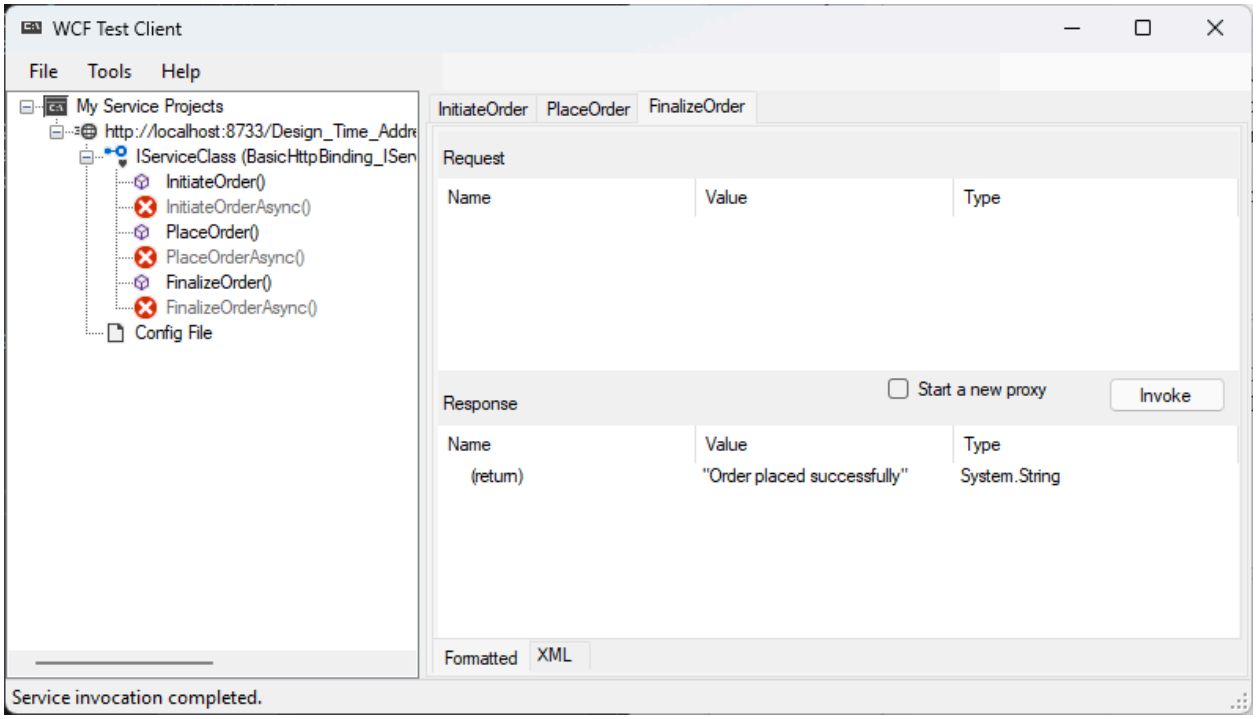
2. Test the Service using Test Client.

=>






WCF Test Client
File
Tools
Help
My Service Projects
http://localhost:8733/Design_Time_Addr
IServiceClass (BasicHttpBinding_ISer
InitiateOrder()
InitiateOrderAsync()
PlaceOrder()
PlaceOrderAsync()
FinalizeOrder()
FinalizeOrderAsync()
Config File
InitiateOrder
PlaceOrder
Request
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
<s:Header>
<Action s:mustUnderstand="1" xmlns="http://schemas.microsoft.com/ws/2005/05/addressing/none
<h:ISBN xmlns:h="http://tempuri.org/">ISBN</h:ISBN>
</s:Header>
<s:Body>
<BookOrder xmlns="http://tempuri.org/">
<Address>25A,manekchok</Address>
<Firstname>John</Firstname>
<Lastname>Mike</Lastname>
<OrderNumber i:nil="true" xmlns:i="http://www.w3.org/2001/XMLSchema-instance" />
<Quantity>2</Quantity>
</BookOrder>
</s:Body>
</s:Envelope>
Response
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
<s:Header>
<h:ISBN xmlns:h="http://tempuri.org/">ISBN</h:ISBN>
</s:Header>
<s:Body>
<BookOrder xmlns="http://tempuri.org/">
<Address>25A,manekchok</Address>
<Firstname>John</Firstname>
<Lastname>Mike</Lastname>
<OrderNumber>12345678</OrderNumber>
<Quantity>2</Quantity>
</BookOrder>
</s:Body>
</s:Envelope>
Formatted
XML
Service invocation completed.


WCF Test Client
File
Tools
Help
My Service Projects
http://localhost:8733/Design_Time_Addr
IServiceClass (BasicHttpBinding_ISer
InitiateOrder()
InitiateOrderAsync()
PlaceOrder()
PlaceOrderAsync()
FinalizeOrder()
FinalizeOrderAsync()
Config File
InitiateOrder
PlaceOrder
FinalizeOrder
Request
Name
Value
Type
Start a new proxy
Invoke
Response
Name
Value
Type
(return)
"Order placed successfully"
System.String
Formatted
XML
Service invocation completed.

3. Develop a Windows Forms Application to Host the service.

=>

Form1.cs :

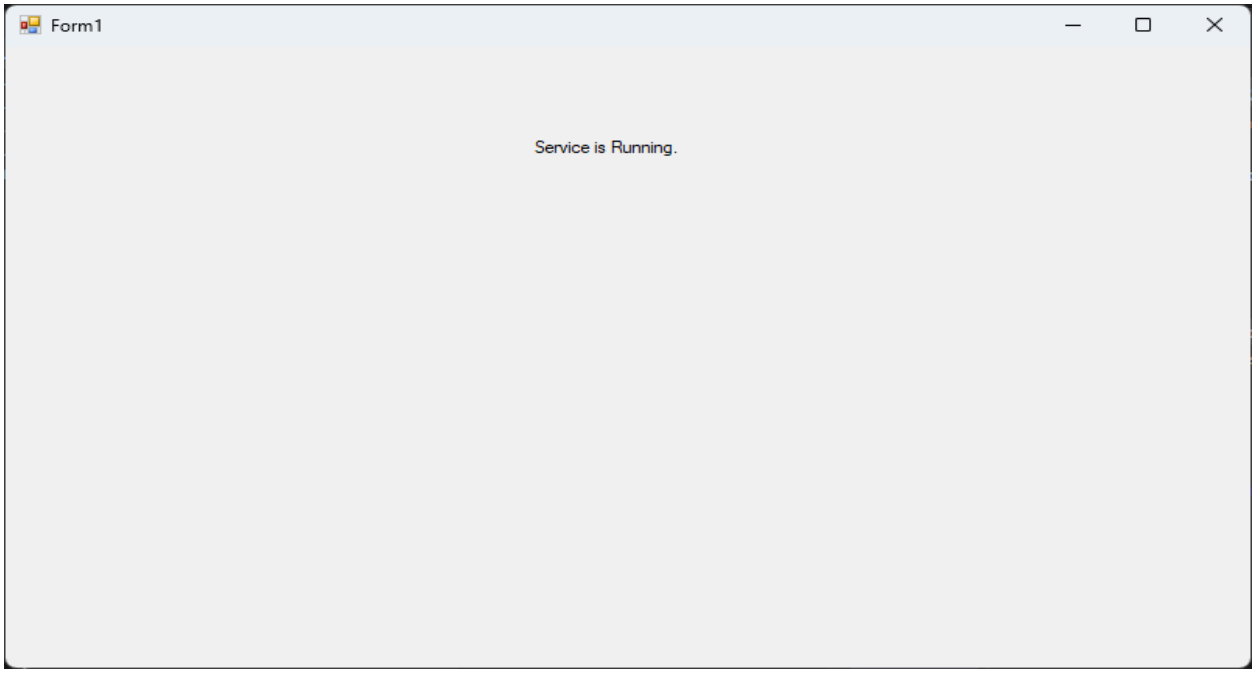
```
using System;
using System.ServiceModel;
using System.Windows.Forms;
namespace WindowsFormsAppHost
{
   public partial class Form1 : Form
   {
      public Form1()
      {
         InitializeComponent();
      }
      ServiceHost sh = null;
      private void Form1_Load(object sender, EventArgs e)
      {
         sh = new ServiceHost(typeof(WcfServiceLibraryBookShop.ServiceClass));
         sh.Open();
         label1.Text = "Service is Running.";
      }
      private void Form1_FormClosing(object sender, FormClosingEventArgs e) {
         sh.Close();
      }
   }
}
```

App.config :

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
   <startup>
      <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.7.2" />
   </startup>
   <system.serviceModel>
      <services>
         <service name="WcfServiceLibraryBookShop.ServiceClass"
behaviorConfiguration="metadatasupport">
            <host>
               <baseAddresses>
                  <add baseAddress="http://localhost:8733/Design_Time_Addresses/" />
```

```
                <add baseAddress="net.pipe://localhost/WCFService"/>
                <add baseAddress="net.tcp://localhost:8080/WCFService"/>
            </baseAddresses>
        </host>
        <endpoint address="tcpmex" binding="mexTcpBinding"
contract="IMetadataExchange"/>
        <endpoint address="namedpipemex" binding="mexNamedPipeBinding"
contract="IMetadataExchange"/>
        <endpoint address="" binding="wsHttpBinding"
contract="WcfServiceLibraryBookShop.IServiceClass" />
      </service>
    </services>
    <behaviors>
      <serviceBehaviors>
        <behavior name="metadatasupport">
          <serviceMetadata httpGetEnabled="False" httpGetUrl=""/>
        </behavior>
      </serviceBehaviors>
    </behaviors>
  </system.serviceModel>
</configuration>
```

Output :



4. Develop a Windows Forms Application to Consume the service.

=>

Form1.cs :

```
using System;
using System.Windows.Forms;
namespace WindowsFormsAppBookShop{
   public partial class Form1 : Form{
      public Form1(){   InitializeComponent();  }
      private void Form1_Load(object sender, EventArgs e){}
      private void button1_Click(object sender, EventArgs e)
      {
         TCP.ServiceClassClient client = new TCP.ServiceClassClient();
         label7.Text = client.InitiateOrder();
         TCP.BookOrder val = new TCP.BookOrder();
         val.ISBN = textBox1.Text;
         val.Quantity=int.Parse(textBox2.Text);
         val.Firstname = textBox3.Text;
         val.Lastname = textBox4.Text;
         val.Address = textBox5.Text;
         TCP.BookOrder reply = ((TCP.IServiceClass)client).PlaceOrder(val);
         textBox6.Text = reply.OrderNumber;
         label8.Text = client.FinalizeOrder();
         client.Close();
      }
   }
}
```

Output :